МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-13

Выполнил
Бондарчук А.Ю,
студент группы
ПО-13

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«_____k ___2017 г.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1.

2) Заводы по производству автомобилей. Реализовать возможность создавать автомобили различных типов на различных заводах.

Код программы

```
from abc import ABC, abstractmethod

#Продукты
class Car(ABC):
    @abstractmethod
    def get_specs(self):
        pass

    def
public_def_only_for_public_def_def_pylint_live_me_along(self):

print("\npublic_def_only_for_public_def_def_pylint_live_me_along\n")

class Sedan(Car):
    def __init__(self, brand):
        self.brand = brand

    def get_specs(self):
        return f"{self.brand} Седан: 4 двери, комфортная подвеска?"

class Jeep(Car):
    def __init__(self, brand):
        self.brand = brand

    def get_specs(self):
        return f"{self.brand} Внедорожник: Полный привод, высокий"

class Hatchback(Car):
    def __init__(self, brand):
        self.brand = brand

    def get_specs(self):
        return f"{self.brand} Хэтчбек: Компактный, 92"

#Абстрактная фабрика
class CarFactory(ABC):
    @abstractmethod
    def create_sedan(self) -> Sedan:
        pass

    @abstractmethod
    def create_jeep(self) -> Jeep:
        pass

    @abstractmethod
    def create_hatchback(self) -> Hatchback:
        pass

#Конкретные фабрики
class VazFactory(CarFactory):
    def create_sedan(self) -> Sedan:
        return Sedan("Lada Vesta")
```

```
        def create_jeep(self) -> Jeep:
            return Jeep("VAZ PATRIOT!")

        def create_hatchback(self) -> Hatchback:
            return Hatchback("Lada XRAY")

    class RenaultFactory(CarFactory):
        def create_sedan(self) -> Sedan:
            return Sedan("Renault LoganXMan")

        def create_jeep(self) -> Jeep:
            return Jeep("Renault Пыльник")

        def create_hatchback(self) -> Hatchback:
            return Hatchback("Renault Prprpr")

    def obertka(factory: CarFactory):
        print("\nРаботает завод:", factory.__class__.__name__)
        car1 = factory.create_sedan()
        car2 = factory.create_jeep()
        car3 = factory.create_hatchback()

        print(car1.get_specs())
        print(car2.get_specs())
        print(car3.get_specs())

    if __name__ == "__main__":
        vaz = VazFactory()
        renault = RenaultFactory()

        obertka(vaz)
        obertka(renault)
```

Рисунки с результатами работы программы

```
Работает завод: VazFactory
Lada Vesta Седан: 4 двери, комфортная подвеска?
VAZ PATRIOT! Внедорожник: Полный привод, высокий
Lada XRAY Хэтчбек: Компактный, 92

Работает завод: RenaultFactory
Renault LoganXMan Седан: 4 двери, комфортная подвеска?
Renault Пыльник Внедорожник: Полный привод, высокий
Renault Prprpr Хэтчбек: Компактный, 92
```

Задание 2.

2) Проект «Универсальная электронная карта». В проекте должна быть реализована универсальная электронная карта, в которой есть функции паспорта, страхового полиса, банковской карты и т. д.

Код программы

#ФАСАД

```
class PassportSubsystem:
    def __init__(self):
        self.full_name = "Ибрагим Абдулович"
        self.birth_date = "25.11.2005"
        self.passport_number = "42"
        self.registration = "Улица Бабалабла"

    def verify_identity(self):
```

```
        print("Паспортная система: проверка личности... OK")
        return True

    def public_def_only_for_public_def_def_pylint_live_me_along(self):
        print("\npublic_def_only_for_public_def_def_pylint_live_me_along\n")

class InsuranceSubsystem:
    def __init__(self):
        self.policy_number = "2022345"

    def check_coverage(self, service):
        print(f"Страховая система: проверка покрытия для услуги '{service}'...
Доступно")
        return True

    def public_def_only_for_public_def_def_pylint_live_me_along(self):
        print("\npublic_def_only_for_public_def_def_pylint_live_me_along\n")


class BankSubsystem:
    def __init__(self):
        self.card_number = "220023456789012"
        self.bank_name = "Мбанк"
        self.balance = 5000.0

    def pay(self, amount, merchant):
        if self.balance >= amount:
            self.balance -= amount
            print(f"Банк: оплата {amount} руб. в '{merchant}' выполнена")
            return True
        print("Банк: недостаточно средств")
        return False

    def money_up(self, amount):
        self.balance += amount
        print(f"Банк: счет пополнен на {amount} руб.")
        return self.balance


#Фасад
class UniversalCardFacade:

    def __init__(self, person_id: str):
        self.person_id = person_id
        self.card_number = f"Кар-Точка ->{person_id}"
        self.passport = PassportSubsystem()
        self.insurance = InsuranceSubsystem()
        self.bank = BankSubsystem()
        print(f"\nИнициализация карты {self.card_number} ")
        print("Карта готова к использованию \n")

    def show_info(self):
        print("\n")
        print("УНИВЕРСАЛЬНАЯ ЭЛЕКТРОННАЯ КАР-ТОЧКА")
        print(f"Карта №: {self.card_number}")
        print(f"Владелец: {self.passport.full_name}")
        print(f"Паспорт: {self.passport.passport_number}")
        print(f"Прописка: {self.passport.registration}")
        print(f"Полис: {self.insurance.policy_number}")
        print(f"Банк: {self.bank.bank_name}")
        print(f"Баланс: {self.bank.balance} руб.")

    def pay(self, amount, merchant = "Покупка"):
        print(f"\nОперация: оплата {amount} руб. в '{merchant}'")
        if self.bank.pay(amount, merchant):
            print("оплата прошла успешно")
```

```
            return True

        return False


    def verify_document(self):
        print("\nОперация: подтверждение личности")
        if self.passport.verify_identity():
            print("Личность подтверждена")
            return True
        return False

    def money_up_balance(self, amount: float):
        print("\nОперация: пополнение баланса")
        new_balance = self.bank.money_up(amount)
        print(f"Новый баланс: {new_balance} руб.")
        return new_balance

if __name__ == "__main__":
    my_card = UniversalCardFacade("USER-001")
    my_card.show_info()

    my_card.pay(500, "Магазин")
    my_card.verify_document()
    my_card.money_up_balance(2000)
```

Рисунки с результатами работы программы

```
Инициализация карты Кар-Точка ->USER-001
Карта готова к использованию



УНИВЕРСАЛЬНАЯ ЭЛЕКТРОННАЯ КАР-ТОЧКА
Карта №: Кар-Точка ->USER-001
Владелец: Ибрагим Абдулович
Паспорт: 42
Прописка: Улица Бабалабла
Полис: 2022345
Банк: Мбанк
Баланс: 5000.0 руб.

Операция: оплата 500 руб. в 'Магазин'
Банк: оплата 500 руб. в 'Магазин' выполнена
оплата прошла успешно

Операция: подтверждение личности
Паспортная система: проверка личности... OK
Личность подтверждена

Операция: пополнение баланса
Банк: счет пополнен на 2000 руб.
Новый баланс: 6500.0 руб.
```

Задание 3.

2) Проект «Принтеры». В проекте должны быть реализованы разные модели принтеров, которые выполняют разные виды печати.Код программы

```
#ФАСАД

from abc import ABC, abstractmethod

#Стратегии
class PrintStrategy(ABC):
    @abstractmethod
    def print(self, document_content):
        pass

    def public_def_only_for_public_def_def_pylint_live_me_along(self):

print("\npublic_def_only_for_public_def_def_pylint_live_me_along\n")


class BWPrint(PrintStrategy):

    def print(self, document_content):
        print("Черно-белая печать ")
        print(f"Текст документа: {document_content}")

class ColorPrint(PrintStrategy):
    def print(self, document_content) :
        print("Цветная печать ")
        print(f"Текст документа (цветной): {document_content}")


class ThreeDPrint(PrintStrategy):
    def print(self, document_content):
        # В данном контексте document_content может быть именем 3D-модели
        print("3D печать ")
        print(f"Печать модели: {document_content}")


#Принтеры
class Printer:
    def __init__(self, model, strategy):
        self.model = model
        self._strategy = strategy

    def set_strategy(self, strategy):
        print(f"\nПринтер {self.model} переключен на
{strategy.__class__.__name__}")
        self._strategy = strategy

    def execute(self, document):
        print(f"\nПринтер {self.model} выполняет задание...")
        self._strategy.print(document)

if __name__ == "__main__":
    bw_strategy = BWPrint()
    color_strategy = ColorPrint()
    three_d_strategy = ThreeDPrint()
```

```
office_printer = Printer("Крепыш", bw_strategy)

office_printer.execute("Важный текст ткст")

office_printer.set_strategy(color_strategy)
office_printer.execute("Цветик севицветик")

three_d_printer = Printer("СуперБамбукЛабик", three_d_strategy)
three_d_printer.execute("моделька")

three_d_printer.set_strategy(bw_strategy)
three_d_printer.execute("Чтото там.txt")
```

Рисунки с результатами работы программы

```
Принтер Крепыш выполняет задание...
Черно-белая печать
Текст документа: Важный текст ткст

Принтер Крепыш переключен на ColorPrint

Принтер Крепыш выполняет задание...
Цветная печать
Текст документа (цветной): Цветик севицветик

Принтер СуперБамбукЛабик выполняет задание...
3D печать
Печать модели: моделька

Принтер СуперБамбукЛабик переключен на BWPrint

Принтер СуперБамбукЛабик выполняет задание...
Черно-белая печать
Текст документа: Чтото там.txt
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.